

УДК 004.42

Область знаний по ГРНТИ: 20.00.00 Информатика; 50.00.00 Автоматика. Вычислительная техника

Тип публикации: оригинальная научная статья

МОДЕЛИРОВАНИЕ СВЯЗЕЙ И ХРОНОЛОГИИ СЮЖЕТНОГО ПРОЕКТА В ВЕБ-СИСТЕМЕ STORYDB

Аннотация: Введение. В сюжетном проекте важно хранить не только карточки персонажей, предметов и мест, но и связи между ними, а также события, в которых эти связи меняются. Иначе противоречия обнаруживаются поздно, уже при подготовке текста, сценария или игровой документации. Цель работы - описать модель связей и хронологии, реализованную в веб-системе StoryDB. Материалы и методы. В качестве исходных данных использованы сущности проекта, типизированные отношения, структуры, события и пользовательские настройки визуальной раскладки. Модель построена на реляционной базе данных; поверх нее формируются граф отношений и таймлайн. Результаты. Выделены источники ребер графа, параметры отношений персонажей, типы событий и правила размещения элементов на временной шкале. Показано, что сохранение координат и закрепленных элементов уменьшает разрыв между автоматической раскладкой и ручной правкой пользователя. Заключение. Совместное использование графа и таймлайна делает StoryDB не только справочником объектов, но и инструментом проверки сюжетной логики, поскольку одно изменение данных отражается сразу в нескольких рабочих представлениях и помогает быстрее находить ошибки в связях и временной последовательности.

Ключевые слова: граф связей; таймлайн; сюжетный проект; визуальное моделирование; хронология; информационная система; StoryDB; реляционная база данных.

MODELING RELATIONS AND TIMELINE OF A STORY PROJECT IN THE STORYDB WEB SYSTEM

Abstract: Introduction. A story project has to store not only cards of characters, items, places and organizations, but also relations between them and events in which these relations change. Otherwise inconsistencies are usually found too late, when a text, script or game document is already being prepared. Purpose. The work describes a relation and timeline model implemented in the StoryDB web system. Materials and methods. The input data include project entities, typed relations, structures, events and user settings of visual layout. The model is based on a relational database; a relation graph and a timeline are generated over the same data. Results. The article identifies sources of graph edges, parameters of character relations, event types and rules for placing timeline elements. Saving coordinates and pinned elements reduces the gap between automatic layout and manual editing. Conclusion. The combined graph and timeline turn StoryDB from a simple object catalog into a tool for checking story logic.

Keywords: relation graph; timeline; story project; visual modeling; chronology; information system; StoryDB; relational database.

Введение

Крупный сюжетный проект быстро перестает быть набором независимых карточек. Один персонаж входит в организацию, получает предмет, покидает место, участвует в событии и после этого меняет статус. Та же сцена может одновременно затрагивать территорию, группу персонажей и несколько предметов. Поэтому для StoryDB важна не только запись «кто есть кто», но и проверяемая сеть зависимостей между сущностями.

При работе в текстовых файлах такие зависимости обычно фиксируются заметками: часть сведений попадает в описание персонажа, часть - в план глав, часть - в отдельную таблицу. На небольшом проекте этого достаточно. При росте числа объектов ручная сверка начинает сбоить: связь уже есть в одном месте, но событие, которое ее объясняет, отсутствует; предмет числится у двух владельцев; организация имеет структуру, но не имеет связей с ее участниками.

Цель работы - разработать и описать модель, которая связывает объектный граф StoryDB с временной линией проекта. Для этого в статье рассматриваются источники ребер, параметры отношений, хранение пользовательской раскладки, типы событий и способ совместного использования графового и хронологического представлений.

Теоретическая часть опирается на работы по графам знаний и графовым моделям, где акцент сделан на явных идентификаторах сущностей, типизированных связях и повторном использовании данных в разных представлениях [6; 8; 9; 11; 12]. В StoryDB эти идеи применяются не как готовая семантическая платформа. Они использованы как инженерный принцип: одна сущность должна участвовать в карточке, структуре, графе и таймлайне без повторного ввода.

Ближайшие аналоги решают задачу частями. Редакторы диаграмм удобны для разовой схемы, wiki-системы хранят текст и ссылки, планировщики таймлайнов показывают последовательность событий, а интеллект-карты помогают разложить идеи по темам. Проблема для сюжетного проекта возникает на стыке этих режимов: изменение в событии должно отразиться на отношениях объектов, а изменение отношений должно быть видно во временном контексте.

Новизна предлагаемого решения состоит в том, что связи и хронология описываются как две проекции одной базы StoryDB. Узлы графа соответствуют объектам проекта, ребра формируются из отношений, владения, принадлежности и структур, а события фиксируют моменты или периоды изменения состояния. За счет этого пользователь работает не с отдельной нарисованной схемой, а с визуальным слоем поверх предметной модели.

Исследования динамических графов и таймлайнов показывают, что для восприятия важны не только состав элементов, но и плавность перехода между состояниями [1; 2; 3]. Поэтому в StoryDB автоматическая раскладка не должна каждый раз «перерисовывать мир» заново. Если пользователь закрепил важный узел или вручную развел конфликтующие элементы, система сохраняет эту правку и учитывает ее при последующих перестроениях.

Графовая модель StoryDB строится поверх реляционной схемы. Такой выбор оставляет обычные механизмы целостности данных и при этом дает несколько способов просмотра одной информации: общий граф связей, иерархии организаций и территорий, временной срез проекта и таймлайн событий. Для автора это выглядит как разные рабочие экраны, но для системы это один набор идентификаторов и отношений.

Материалы и методы

В качестве узлов графа используются персонажи, предметы, места и организации. У каждого узла есть постоянный идентификатор, название, тип, изображение и набор признаков, которые можно вывести на экран. Ребра не вводятся только вручную: они собираются из отношений персонажей, привязок предметов, структуры организаций, территориальной принадлежности и связей событий с объектами.

Для отношений персонажей сохраняются сила, напряженность, направленность и признак двусторонности. Эти поля нужны, потому что одинаковая подпись связи не всегда означает одинаковое содержание. Например, союз и зависимость могут выглядеть как положительная связь, но по-разному влияют на конфликтность сцены и на отображение пары персонажей в анализаторе.

Структуры выделены в отдельный слой. Они описывают иерархию организации, состав территории, систему подчинения, родословную или любую другую внутреннюю композицию. Узел структуры может быть связан с объектом или записью каталога, а назначение объекта в структуру затем используется при построении графа.

В модели различаются несколько типов отношений. Персональные связи описывают взаимодействие персонажей; отношения владения показывают переход предметов; территориальные связи фиксируют местоположение; структурные связи показывают включение элемента в группу. Поэтому в базе хранится не только пара объектов, но и тип отношения, источник, направление и дополнительные параметры.

Хранение построено на реляционной модели [4]. Объекты и визуальные представления разделены: изменение координаты узла не меняет сам объект, а удаление или переименование связи проверяется через идентификаторы. Это особенно важно для таймлайна, где одно событие может быть связано с несколькими объектами.

Временная модель разделяет события на точку, длительность, главу и эпоху. Точка фиксирует эпизод, длительность задает интервал, глава отражает композиционное деление текста, а эпоха обозначает крупный фон. Между событиями допускаются связи предшествования, причины,

одновременности и включения; эти связи помогают отличить простой список эпизодов от логической цепочки.

При раскладке таймлайна система сначала рассчитывает положение элементов по числовым значениям начала и окончания, затем распределяет их по дорожкам. Закрепленные пользователем элементы не сдвигаются без необходимости. Такой подход согласуется с задачами динамической визуализации, где резкая перестройка ухудшает ориентацию пользователя [1; 2].

Программная реализация соотнесена с подходами к модульной архитектуре и микросервисным системам [5; 7; 10]. В StoryDB это выражено не в полном разделении сервиса на независимые приложения, а в более практичном решении: модули карточек, графа, таймлайна и структур имеют собственные сценарии обработки, но работают с общей проектной базой.

Этапы формирования визуальной модели StoryDB приведены в таблице 1; обобщенная схема данных и фрагмент модуля хронологии показаны на рисунках 1 и 2.

Таблица 1 - Этапы формирования визуальной модели StoryDB

Table 1 - Main modules/stages of StoryDB

Этап	Данные	Результат
Нормализация объектов	Карточки персонажей, предметов, мест и организаций	Единый набор узлов графа проекта
Формирование связей	Отношения персонажей, владение, территории, иерархии и структуры	Набор ребер с типом, категорией и дополнительными параметрами
Учет времени	События, периоды, изменения состояния объектов и связи partOf	Контекстная модель проекта для выбранного события или периода
Раскладка графа	Узлы, ребра, сохраненные координаты, закрепленные элементы	Визуальная схема отношений с возможностью ручной корректировки
Раскладка таймлайна	Точки, длительности, главы, эпохи и причинно-временные связи	Хронологическая диаграмма без пересечения основных элементов

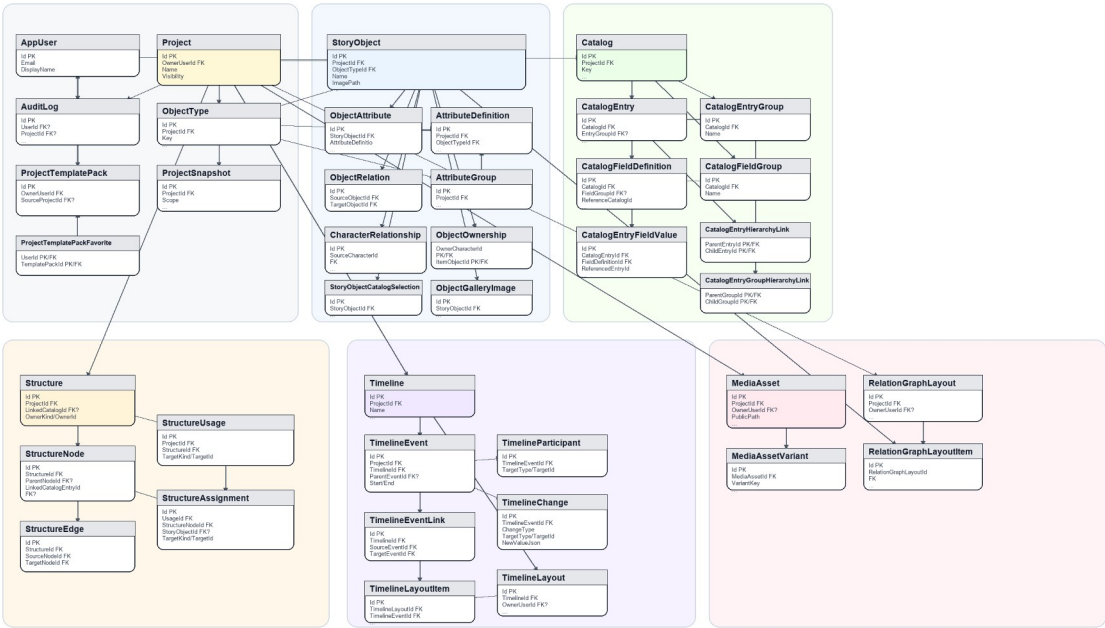


Рисунок 1. Обобщенная схема данных StoryDB

Figure 1. General data schema of StoryDB

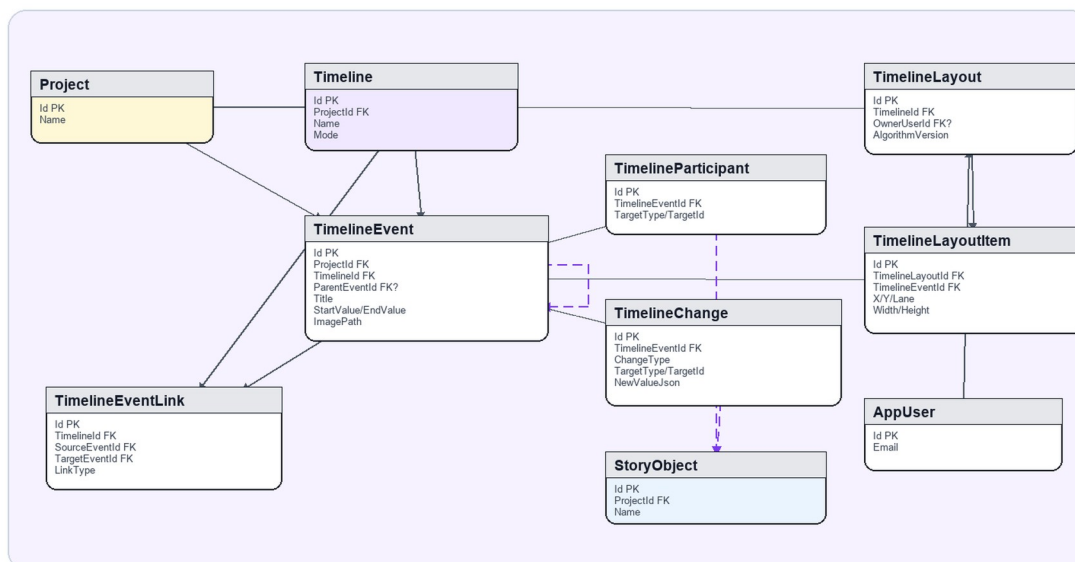


Рисунок 2. Фрагмент схемы данных модуля хронологии
Figure 2. Fragment of the timeline module data schema

Результаты и обсуждение

В реализованном модуле хронологии событие хранит тип, начальное и конечное значение, связи с объектами и связи с другими событиями. Если дата задана точно, элемент ставится в соответствующее место шкалы. Если дата условная, используется относительный порядок внутри проекта. Это позволяет вести как строгую историческую линию, так и черновую авторскую хронологию.

Для точечного события вычисляется координата на оси времени, для длительности - ширина интервала. Главы и эпохи размещаются отдельно, поскольку они описывают не столько факт внутри мира, сколько уровень композиции или общий временной фон. Такое разделение уменьшает путаницу между «когда произошло» и «в каком фрагменте текста это показывается».

Пересечения элементов устраняются распределением по дорожкам. Сначала размещаются крупные интервалы, затем события с зависимостями, после этого система подбирает свободные дорожки для независимых элементов. В результате таймлайн остается читаемым даже тогда, когда в одном отрезке проекта сосредоточено несколько сцен и переходов состояния.

Для графа отношений главным результатом стала поддержка ручной раскладки. Пользователь может передвинуть узел, изменить размер области, закрепить важные элементы и вернуться к этой схеме позже. В базе для этого сохраняются координаты, размеры, ключ графа, версия алгоритма и признак устаревания раскладки.

Совмещение графа и таймлайна дает два режима проверки. В статическом режиме видна сеть объектов: кто с кем связан, кому принадлежит предмет, в какую структуру входит объект. Во временном режиме проверяется, когда эта связь появилась, исчезла или стала причиной другого события.

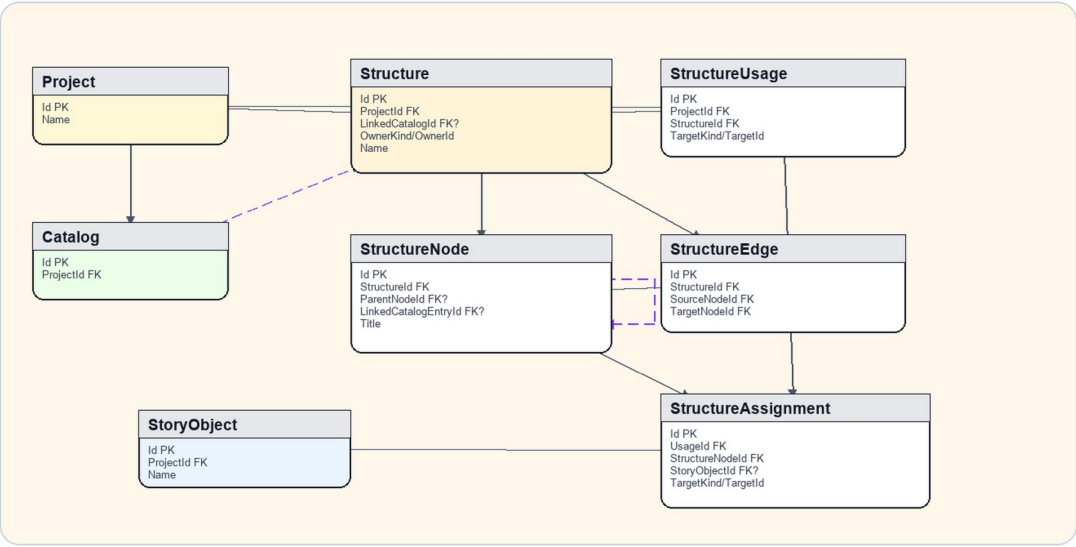
Одна и та же база данных формирует несколько представлений. Общий граф показывает связи между сущностями, структурное представление подчеркивает иерархию, а таймлайн выводит события, интервалы и причинные переходы. Пользователь не переписывает сведения для каждого экрана; он уточняет модель, а визуальные формы собираются из нее.

Практическая польза модели проявляется при поиске противоречий. Например, персонаж включен в организацию, но событие вступления не задано; предмет указан у нового владельца, но событие передачи стоит позже сцены использования; территория показана как место события, хотя в этот период она относится к другой структуре. Такие ошибки заметнее, когда граф и хронология проверяются вместе.

Модуль таймлайна также помогает отделять внутреннюю историю мира от композиции произведения. Эпоха может охватывать большой период, глава - участок текста, длительное событие

- состояние персонажа, а точка - конкретный эпизод. Такая схема подходит и для литературного проекта, и для игровой базы данных, где автору нужны разные масштабы описания.

Ограничение подхода связано с качеством заполнения данных. Если автор не фиксирует смену владельца, не привязывает событие к участникам или оставляет структуру неполной, система не сможет вывести противоречие автоматически. Поэтому следующий шаг для StoryDB - подсказки неполноты: предупреждения о связи без события, событии без участников и пересечении несовместимых состояний.



Фрагмент структуры иерархий, а также интерфейсы анализа связей, работы с таймлайном и управления структурами приведены на рисунках 3-6.

Рисунок 3. Фрагмент схемы данных структур и иерархий
Figure 3. Fragment of the structures and hierarchies data schema

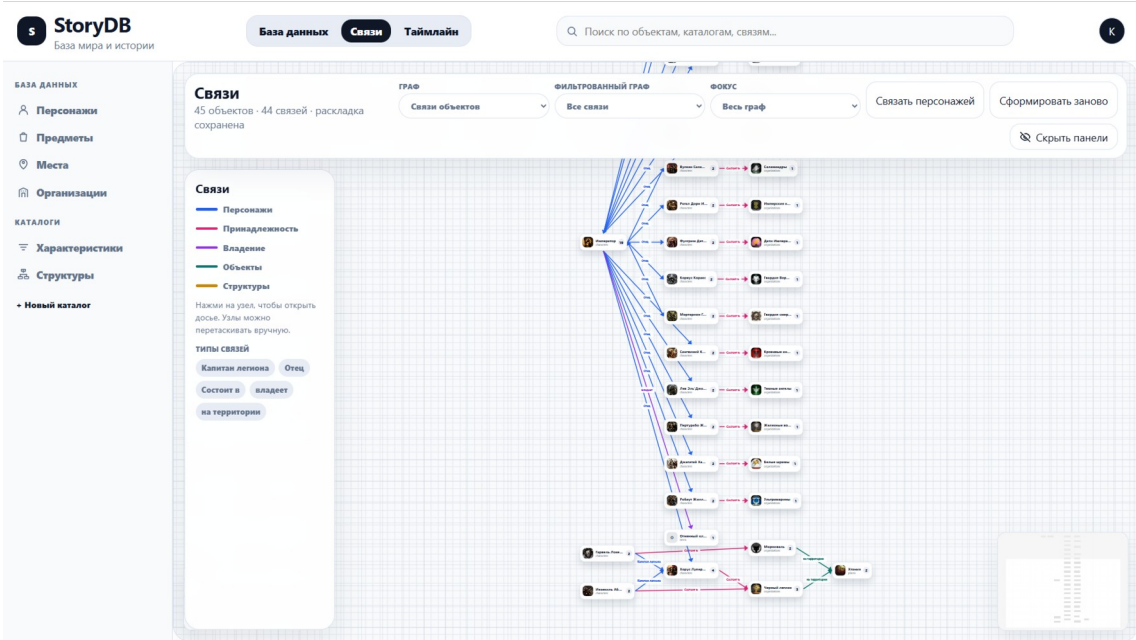


Рисунок 4. Интерфейс анализа связей объектов
Figure 4. Interface for analyzing object relations

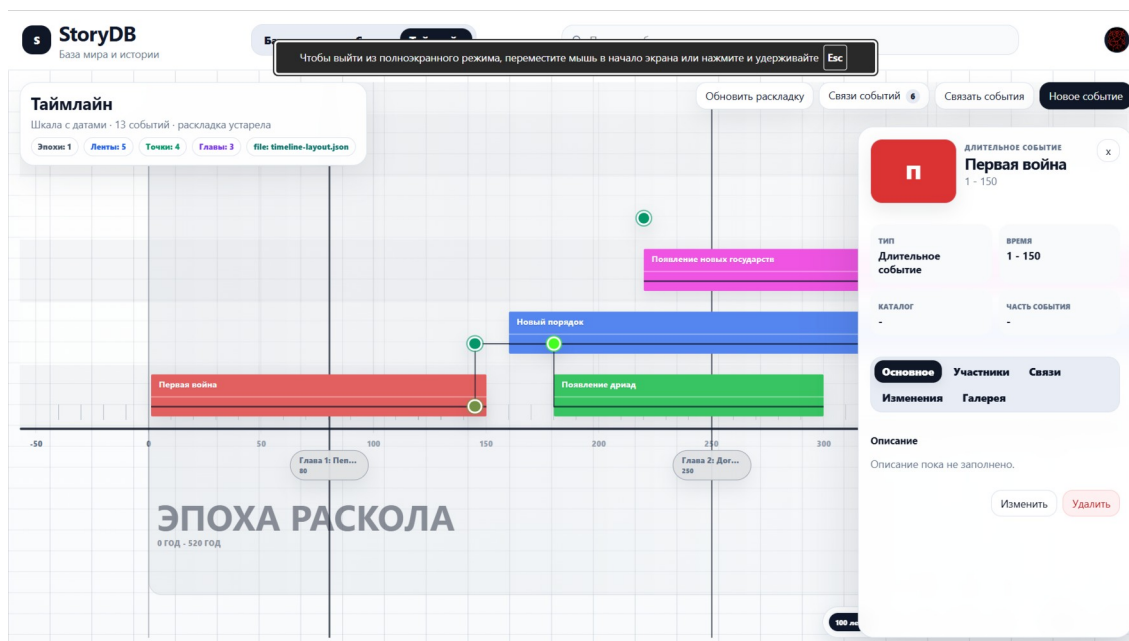


Рисунок 5. Интерфейс работы с таймлайном проекта
Figure 5. Interface for working with the project timeline

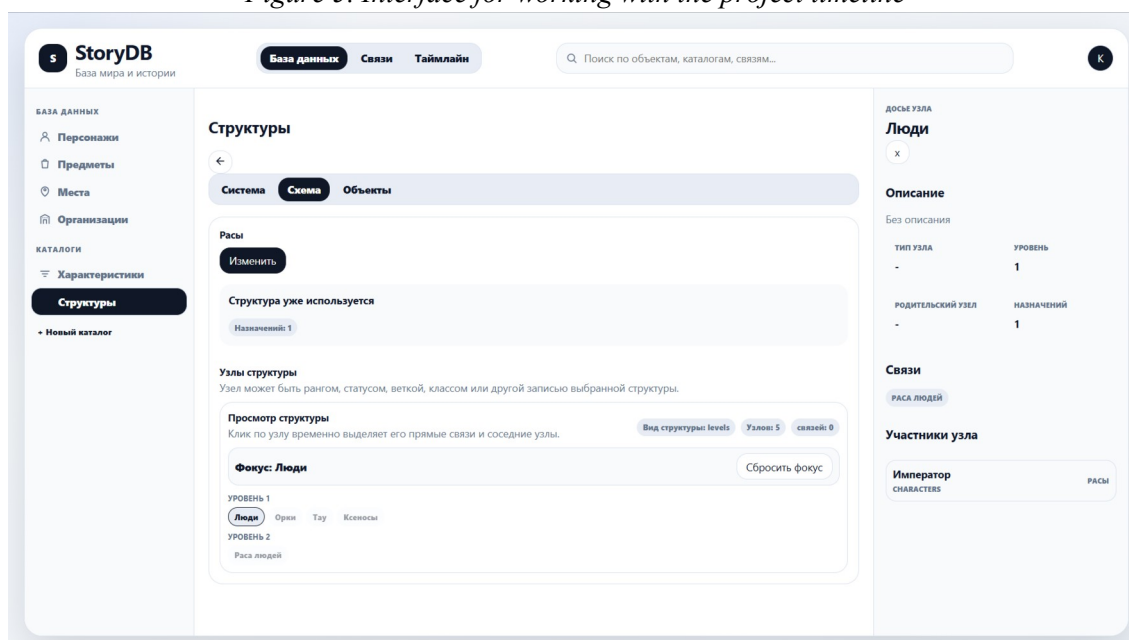


Рисунок 6. Интерфейс управления структурами проекта
Figure 6. Interface for managing project structures

Заключение

В результате работы в StoryDB получена модель, где сюжетный проект рассматривается как набор объектов, связей и событий. Граф отвечает за смысловые зависимости между сущностями, а таймлайн показывает, когда эти зависимости появляются, меняются или становятся причиной дальнейших событий.

Для реализации модели выделены источники ребер, типы отношений, параметры персональных связей, типы событий и данные визуальной раскладки. Сохранение координат и закрепленных элементов позволяет совмещать автоматическое построение схемы с ручной редакторской правкой.

Предложенный подход полезен тем, что не требует вести отдельную диаграмму и отдельную хронологию. Оба представления собираются из одной базы, поэтому уточнение карточки, структуры или события сразу влияет на анализ проекта.

Дальнейшая работа должна быть направлена на формальные проверки: поиск недостающих событий перехода, конфликтующих владельцев, нарушений причинной последовательности и несовместимых состояний объекта в один период.

Литература

1. Bach B., Pietriga E., Fekete J.D. GraphDiaries: Animated Transitions and Temporal Navigation for Dynamic Networks // *IEEE Transactions on Visualization and Computer Graphics*. 2014. Vol. 20. No. 5. P. 740-754. DOI: 10.1109/TVCG.2013.254.
2. Beck F., Burch M., Diehl S., Weiskopf D. A Taxonomy and Survey of Dynamic Graph Visualization // *Computer Graphics Forum*. 2017. Vol. 36. No. 1. P. 133-159. DOI: 10.1111/cgf.12791.
3. Brehmer M., Lee B., Bach B., Riche N.H., Munzner T. Timelines Revisited: A Design Space and Considerations for Expressive Storytelling // *IEEE Transactions on Visualization and Computer Graphics*. 2017. Vol. 23. No. 9. P. 2151-2164. DOI: 10.1109/TVCG.2016.2614803.
4. Chen P.P.S. The Entity-Relationship Model - Toward a Unified View of Data // *ACM Transactions on Database Systems*. 1976. Vol. 1. No. 1. P. 9-36. DOI: 10.1145/320434.320440.
5. Di Francesco P., Lago P., Malavolta I. Architecting with microservices: A systematic mapping study // *Journal of Systems and Software*. 2019. Vol. 150. P. 77-97. DOI: 10.1016/j.jss.2019.01.001.
6. Hogan A., Blomqvist E., Cochez M., d'Amato C., de Melo G., Gutierrez C., Kirrane S., Gayo J.E.L., Navigli R., Neumaier S., Ngomo A.C.N., Polleres A., Rashid S.M., Rula A., Schmelzeisen L., Sequeda J., Staab S., Zimmermann A. Knowledge Graphs // *ACM Computing Surveys*. 2021. Vol. 54. No. 4. Article 71. DOI: 10.1145/3447772.
7. Jamshidi P., Pahl C., Mendonca N.C., Lewis J., Tilkov S. Microservices: The Journey So Far and Challenges Ahead // *IEEE Software*. 2018. Vol. 35. No. 3. P. 24-35. DOI: 10.1109/MS.2018.2141039.
8. Ji S., Pan S., Cambria E., Marttinen P., Yu P.S. A Survey on Knowledge Graphs: Representation, Acquisition, and Applications // *IEEE Transactions on Neural Networks and Learning Systems*. 2022. Vol. 33. No. 2. P. 494-514. DOI: 10.1109/TNNLS.2021.3070843.
9. Noy N., Gao Y., Jain A., Narayanan A., Patterson A., Taylor J. Industry-scale knowledge graphs: Lessons and challenges // *Communications of the ACM*. 2019. Vol. 62. No. 8. P. 36-43. DOI: 10.1145/3331166.
10. Pahl C., Jamshidi P. Microservices: A Systematic Mapping Study // *Proceedings of the 6th International Conference on Cloud Computing and Services Science*. 2016. P. 137-146. DOI: 10.5220/0005785500001259.
11. Peng C., Feng X., Naseriparsa M., Osborne F. Knowledge Graphs: Opportunities and Challenges // *Artificial Intelligence Review*. 2023. Vol. 56. P. 13071-13102. DOI: 10.1007/s10462-023-10465-9.
12. Zhou J., Cui G., Hu S., Zhang Z., Yang C., Liu Z., Wang L., Li C., Sun M. Graph neural networks: A review of methods and applications // *AI Open*. 2020. Vol. 1. P. 57-81. DOI: 10.1016/j.aiopen.2021.01.001.

References

1. Bach, B., Pietriga, E., & Fekete, J. D. (2014). GraphDiaries: Animated transitions and temporal navigation for dynamic networks. *IEEE Transactions on Visualization and Computer Graphics*, 20(5), 740-754. <https://doi.org/10.1109/TVCG.2013.254>
2. Beck, F., Burch, M., Diehl, S., & Weiskopf, D. (2017). A taxonomy and survey of dynamic graph visualization. *Computer Graphics Forum*, 36(1), 133-159. <https://doi.org/10.1111/cgf.12791>
3. Brehmer, M., Lee, B., Bach, B., Riche, N. H., & Munzner, T. (2017). Timelines revisited: A design space and considerations for expressive storytelling. *IEEE Transactions on Visualization and Computer Graphics*, 23(9), 2151-2164. <https://doi.org/10.1109/TVCG.2016.2614803>
4. Chen, P. P. S. (1976). The Entity-Relationship Model - Toward a unified view of data. *ACM Transactions on Database Systems*, 1(1), 9-36. <https://doi.org/10.1145/320434.320440>
5. Di Francesco, P., Lago, P., & Malavolta, I. (2019). Architecting with microservices: A systematic mapping study. *Journal of Systems and Software*, 150, 77-97. <https://doi.org/10.1016/j.jss.2019.01.001>
6. Hogan, A., Blomqvist, E., Cochez, M., d'Amato C., de Melo, G., Gutierrez, C., Kirrane, S., Gayo, J. E. L., Navigli, R., Neumaier, S., Ngomo, A. C. N., Polleres, A., Rashid, S. M., Rula, A., Schmelzeisen L.,

- Sequeda, J., Staab, S., & Zimmermann, A. (2021). Knowledge graphs. *ACM Computing Surveys*, 54(4), Article 71. <https://doi.org/10.1145/3447772>
7. Jamshidi, P., Pahl, C., Mendonca, N. C., Lewis, J., & Tilkov, S. (2018). Microservices: The journey so far and challenges ahead. *IEEE Software*, 35(3), 24-35. <https://doi.org/10.1109/MS.2018.2141039>
8. Ji, S., Pan, S., Cambria, E., Marttinen, P., & Yu, P. S. (2022). A survey on knowledge graphs: Representation, acquisition, and applications. *IEEE Transactions on Neural Networks and Learning Systems*, 33(2), 494-514. <https://doi.org/10.1109/TNNLS.2021.3070843>
9. Noy, N., Gao, Y., Jain, A., Narayanan, A., Patterson, A., & Taylor, J. (2019). Industry-scale knowledge graphs: Lessons and challenges. *Communications of the ACM*, 62(8), 36-43. <https://doi.org/10.1145/3331166>
10. Pahl, C., & Jamshidi, P. (2016). Microservices: A systematic mapping study. In *Proceedings of the 6th International Conference on Cloud Computing and Services Science* (pp. 137-146). <https://doi.org/10.5220/0005785500001259>
11. Peng, C., Feng, X., Naseriparsa, M., & Osborne, F. (2023). Knowledge graphs: Opportunities and challenges. *Artificial Intelligence Review*, 56, 13071-13102. <https://doi.org/10.1007/s10462-023-10465-9>
12. Zhou, J., Cui, G., Hu, S., Zhang, Z., Yang, C., Liu, Z., Wang, L., Li, C., & Sun, M. (2020). Graph neural networks: A review of methods and applications. *AI Open*, 1, 57-81. <https://doi.org/10.1016/j.aiopen.2021.01.001>